

Tugas Minggu 2: Rancangan Konsep Strategi Optimasi Komputasi Awan



Disusun Oleh:

I Dewa Made Satya Raditya	5027231051
Ahmad Wildan Fawwaz	5027241001
Muhammad Rakha Hananditya Rauf	5027241015
Theodorus Aaron Ugraha	5027241056
M. Hikari Reiziq Rakhmadinta	5027241079

Mata Kuliah : Strategi Optimasi Komputasi Awan

Kelas : C

Dosen Pengampu : Dr. Ir. Henning Titi Ciptaningtyas, S.Kom., M.Kom.

Institut Teknologi Sepuluh Nopember

Surabaya

2026

Jenis Task dan Workload yang Dipilih beserta Dataset Ujicoba

a. Jenis Task yang Dipilih

Proyek ini menggunakan *independent tasks* yang dibagi menjadi dua kelas sesuai skenario *fog-cloud* pada *paper* utama. Pemisahan perilaku *dependent* dan *independent task* merupakan salah satu tantangan penjadwalan *fog-cloud* yang diidentifikasi pada *paper* tersebut.

- **Task *delay-sensitive***

Berasal dari aliran data *time series* perangkat IoT berupa hasil pembacaan sensor dengan batas waktu (*deadline*) ketat dan prioritas eksekusi tinggi. Task ini diarahkan ke *fog node* agar diproses dekat dengan sumber data selaras dengan hierarki tiga lapis (*device, fog, dan cloud*) serta arsitektur komputasi untuk aplikasi *real-time*.

Paper utama mencatat preferensi studi IoT pada instruksi singkat, penegakan *deadline* ketat (misalnya pada layanan kesehatan), dan minimalisasi pelanggaran latensi.

- **Task *computation-intensive***

Berupa pekerjaan analitik *batch* dengan batas waktu lebih longgar, seperti agregasi data historis dan deteksi anomali. Task ini dialokasikan ke VM di *cloud* karena membutuhkan kapasitas komputasi besar dan pemrosesan data masif.

b. Karakteristik Workload yang Disimulasikan

- **Kedatangan task bersifat dinamis dan stokastik:** Mengikuti rekomendasi kebijakan adaptif dinamis pada *paper* utama, pola kedatangan acak dimodelkan menggunakan proses Poisson (*Poisson process*).
- **Heterogenitas atribut task:** Setiap task memiliki variasi ukuran komputasi (dalam satuan MI), ukuran *output*, kebutuhan unit pemrosesan (*Processing Elements / PE*), dan *deadline* yang harus dipenuhi oleh kapasitas VM tujuan.
- **Variasi densitas beban:** Jumlah task diskalakan antar skenario pengujian guna mengevaluasi ketahanan penjadwal dari kondisi beban rendah hingga beban puncak (*burst arrivals*).

c. Dataset yang Diujicoba

Evaluasi dilakukan melalui simulasi berbasis **CloudSim** dan **iFogSim**, menggunakan kombinasi data riil dan beban sintetis untuk mengatasi limitasi ketiadaan data riil pada beberapa studi terdahulu:

1. Intel Lab Data

- **Deskripsi:** Log *time series* dari 54 sensor node berisi metrik suhu, kelembapan, intensitas cahaya, dan tegangan (> 2 juta rekaman).
- **Peran:** Sumber task *delay-sensitive* di sisi *fog node*, dengan *timestamp* riil sebagai acuan interval kedatangan task.

2. GoCJ (Google Cloud Jobs)

- **Deskripsi:** Dataset ukuran pekerjaan berskala instruksi yang diturunkan dari *trace* kluster Google melalui simulasi Monte Carlo, berformat kompatibel dengan CloudSim.
- **Peran:** Sumber task *computation-intensive* pada VM cloud.

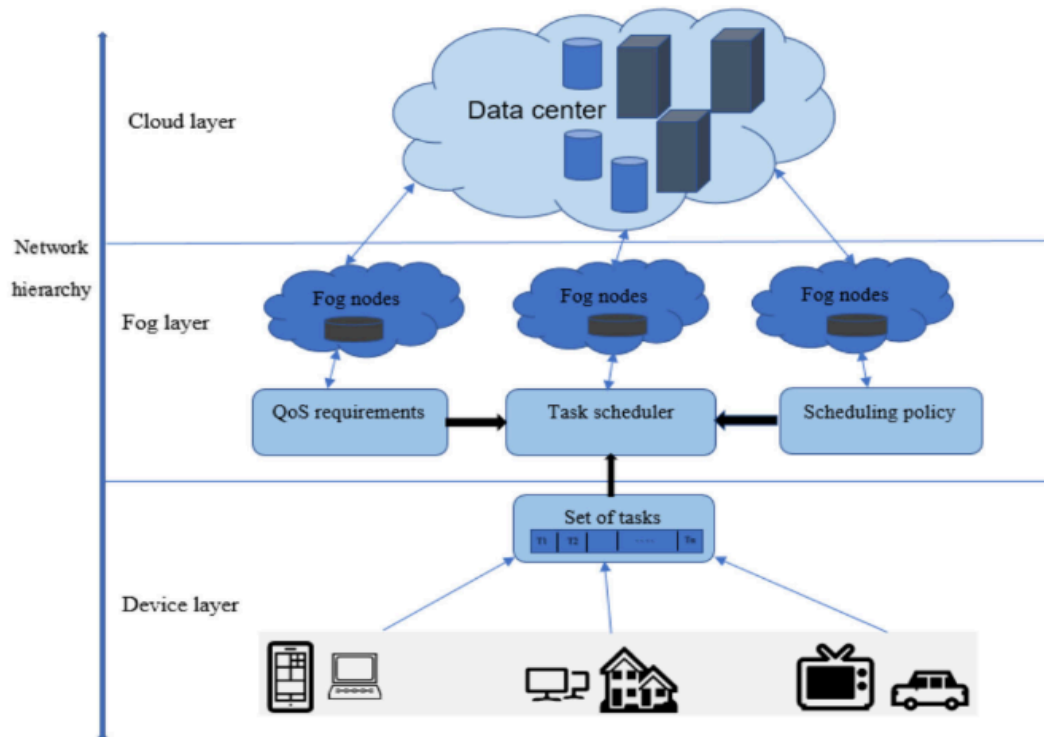
3. Workload Sintetis

- **Deskripsi:** Pembangkitan data parameterik langsung via konfigurasi simulator.
- **Peran:** Menguji stabilitas dan skalabilitas algoritma terhadap variasi laju kedatangan serta lonjakan beban ekstrem (*bursts*).

d. Alasan Pemilihan

- **Kompatibilitas simulator:** Format dataset selaras dengan ekosistem evaluasi CloudSim dan iFogSim yang mendominasi literatur *fog-cloud*.
- **Mitigasi bias sintetis:** Menggabungkan dataset riil dan sintetis menjawab kritik atas studi-studi terdahulu yang bergantung penuh pada data buatan.
- **Aksesibilitas dan adaptabilitas:** Keduanya merupakan dataset publik dengan format terstruktur yang mudah dipetakan menjadi atribut task pada lingkungan simulasi.

Data Center, Host, VM, dan Task/Cloudlet



- a. **Konsep Hierarki Jaringan:** Arsitektur yang dirancang akan menggunakan topologi tiga lapis:
 1. Lapisan Perangkat (menghasilkan *task*),
 2. Lapisan *Fog* (berisi *fog nodes* dan *scheduler* untuk respons cepat),
 3. Lapisan *Cloud* (pusat data/*data center* untuk penyimpanan massal dan komputasi berat).
- b. **Manajemen VM:** Untuk mengoptimalkan efisiensi energi di *data center*, strategi ini akan menerapkan teknik penempatan Mesin Virtual (VM) awal disesuaikan dengan kapasitas *host* dan kebutuhan beban kerja, serta menerapkan migrasi VM secara langsung (*live migration*) untuk menyeimbangkan beban. Selain itu, *host* fisik yang berstatus menganggur (*idle*) dapat dimatikan (*shutdown* atau dialihkan ke *sleep mode*) tanpa melanggar *Service-Level Agreement* (SLA).
- c. **Rancangan Implementasi:** Untuk mengeksekusi *task* atau *cloudlet* secara lebih ringan, VM di level *host* dapat menjalankan sistem berbasis kontainer (seperti Docker) sebagai unit penjadwalan tugas. Jika task berupa deteksi anomali paket jaringan ringan (seperti *rules Intrusion Detection System*), task dapat diprioritaskan untuk dieksekusi di *fog nodes* selama kapasitas dan kondisi antriannya memungkinkan dalam pemenuhan *deadline*. Sedangkan *task* yang memuat log agregasi dalam jumlah masif dapat dikirimkan (*offloading*) ke VM di lapisan *cloud*.

untuk analitik tingkat lanjut. Keputusan *offloading* nantinya akan mempertimbangkan ukuran *task*, panjang antrian, serta perkiraan waktu transfer dan pemrosesan.

Sumber Daya Homogen dan Heterogen (Homogeneous and Heterogeneous Resources)

- a. **Konsep dan Landasan Teori:** Sumber daya homogen merupakan sumber daya yang memiliki spesifikasi yang sama atau setara, sedangkan sumber daya heterogen merupakan sumber daya yang memiliki spesifikasi berbeda, seperti kecepatan CPU, kapasitas RAM, dan *bandwidth*. Lingkungan komputasi modern beroperasi di atas arsitektur komputasi awan yang heterogen, dengan kemampuan sumber daya yang dapat berbeda dari ujung *fog* ke *cloud*. Perbedaan kemampuan tersebut berpotensi mempengaruhi pembagian pekerjaan karena jumlah *task* yang sama belum tentu menghasilkan beban yang seimbang.
- b. **Penjadwalan di Lingkungan Heterogen:** Strategi penjadwalan harus dirancang untuk sistem komputasi yang heterogen, di mana algoritma penugasan (heuristik atau meta-heuristik) dapat mengidentifikasi *node* mana yang paling efisien secara energi (fitur *Most-Efficient-Server-First*) untuk mengeksekusi *task*.
- c. **Resource Over-commitment:** Untuk memaksimalkan manfaat komputasi dari server fisik yang beragam, proyek ini dapat menerapkan konsep *resource over-commitment*. Konsep ini memungkinkan sistem penjadwal untuk mengalokasikan sumber daya tervirtualisasi dalam jumlah yang lebih besar daripada padanan sumber daya fisik nyatanya, sehingga berpotensi mengurangi biaya operasional dan pemborosan *resource*. Namun, manfaatnya perlu diuji saat permintaan sumber daya meningkat secara bersamaan karena perebutan resource dapat memperpanjang waktu eksekusi dan mengganggu kestabilan layanan.
- d. **Rancangan Implementasi:** Algoritma hibrida (seperti penggabungan *Particle Swarm Optimization* / PSO) akan dikembangkan untuk mengevaluasi *fitness function* dari setiap *task* yang masuk terhadap ketersediaan server heterogen. Pencocokan cerdas ini akan memastikan bahwa beban instruksi komputasi dialokasikan ke *node* yang memiliki kapasitas energi dan kecepatan eksekusi paling optimal. Untuk sumber daya yang homogen di dalam sub-klaster *data center* tertentu, mekanisme antrian sederhana (Round Robin modifikasi) dapat digunakan untuk menyeimbangkan utilisasi.

The Best Practice to Implement

Berdasarkan paper tinjauan literatur tersebut, dapat disimpulkan bahwa *best practice* nya adalah menerapkan pendekatan *Hybrid-Metaheuristic* yang digabungkan dengan optimasi *Multi-Objective* (mempertimbangkan *makespan*, biaya, dan efisiensi energi secara bersamaan) serta didukung oleh kerangka kerja adaptif berbasis *Learning-Based/AI* untuk lingkungan yang terdistribusi (*Cloud-Fog*).

Pendekatan ini dipilih menjadi *best practice* karena mampu menutupi kelemahan dari masing-masing metode tunggal: algoritma konvensional atau heuristik statis terlalu kaku dan mudah terjebak di solusi *suboptimal* saat menghadapi beban kerja yang besar, sementara metaheuristik murni seringkali memerlukan biaya komputasi yang tinggi dan sensitif terhadap *parameter tuning*. Dengan menggabungkannya ke dalam model *hybrid* (seperti memadukan pencarian global *metaheuristic* dengan *local search* atau *machine learning*), sistem dapat beradaptasi secara dinamis terhadap fluktuasi beban kerja *real-time*. Selain itu, integrasi prinsip *fog computing* (mendekatkan komputasi ke pengguna) serta evaluasi multi-objektif akan memastikan proyek Anda tidak hanya mengejar kecepatan eksekusi (*makespan* yang rendah), tetapi juga efisiensi biaya dan penghematan energi yang berkelanjutan, sebagaimana ditekankan sebagai tren masa depan di dalam literatur-literatur tersebut.